

OFFSEC

PYTHON FOR RED TEAM



14

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Tipos de dados	6
Código-fonte 2 – Concatenando variáveis	8
Código-fonte 3 – Operações matemáticas	9
Código-fonte 4 – Estrutura condicional	10
Código-fonte 5 – Estrutura for	11
Código-fonte 6 – Estrutura while	11
Código-fonte 7 – Listas	13
Código-fonte 8 – Funções	13
Código-fonte 9 – Cliente TCP.....	14
Código-fonte 10 – Servidor TCP	15
Código-fonte 11 – Lidando com endianness	16

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 – Acessando o modo interativo do Python.....	5
Comando de prompt 2 – Exemplos de utilização do modo interativo.....	6
Comando de prompt 3 – Acessando o servidor TCP via <i>netcat</i>	15
Comando de prompt 4 – Convertendo o big endian para little endian.....	16

EMANIP

SUMÁRIO

1 PYTHON FOR RED TEAM	5
1.1 Introdução	5
2 SOBRE O PYTHON	5
2.1 Python Interativo	5
2.2 Variáveis e concatenação	6
2.3 Operações matemáticas.....	9
2.4 Estruturas Condicionais.....	10
2.5 Laços de repetição	11
2.6 Listas	12
2.7 Funções.....	13
2.8 Sockets.....	14
3 LIDANDO COM VALORES EM BIG ENDIAN E LITTLE ENDIAN.....	15
REFERÊNCIAS.....	17

1 PYTHON FOR RED TEAM

1.1 Introdução

Neste capítulo vamos fazer uma revisão sobre a linguagem Python do ponto de vista de um desenvolvedor de *exploit*. É esperado que o leitor já tenha conhecimentos de lógica de programação, e, caso já tenha experiência em alguma dessas linguagens, como C, C++ ou Ruby, terá mais facilidade para entender os tópicos subsequentes do conteúdo.

2 SOBRE O PYTHON

De acordo com o site Computer World, o Python é uma linguagem de programação de alto nível e muito versátil. Ela suporta tanto a programação orientada a objetos quanto a programação estruturada. Com Python, você pode acessar bibliotecas nativas que oferecem funcionalidades para desenvolvimento de projetos e implementação de aplicações complexas. Por ser tão versátil, vamos utilizá-la para criar nossos scripts de fundamentos básicos e futuramente nossos *exploits*.

2.1 Python Interativo

Utilizaremos com bastante frequência o Python no modo interativo, em que o interpretador do Python trabalha como se fosse uma shell de Unix, podendo executar comandos de Python interativamente, como, por exemplo, efetuar uma operação de soma ou subtração, elaborar laços de repetição e tudo que o programa oferece. Veja como utilizar esse modo no comando:

```
root@kali:~# python
Python 2.7.18 (default, Apr 20 2020, 20:30:41)
[GCC 9.3.0] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Comando de prompt 1 – Acessando o modo interativo do Python
Fonte: Elaborado pelo autor (2021)

Utilizando o Python interativo é possível fazer várias operações que são utilizadas em um script tradicional, como no exemplo a seguir:

```
>>> a = 1 + 1
>>> b = "Oi"
>>> print a
2
>>> print b
Oi
```

Comando de prompt 2 – Exemplos de utilização do modo interativo
Fonte: Elaborado pelo autor (2021)

2.2 Variáveis e concatenação

Trabalharemos frequentemente com variáveis (nome que se refere a um valor) e, para isso, será necessário ter conhecimento para utilizar cada tipo de dado que pode ser armazenado em uma variável. Veja no código-fonte “Tipos de dados” como utilizar cada um dos tipos de variáveis.

```
#!/usr/bin/python
# tipos.py

# Para executar o script: python tipos.py

tipo_int = 100 # Tipo Inteiro
tipo_string = "Testando" # Tipo Texto
tipo_float = 1.1 # Tipo Float
tipo_bool = False # Tipo Boolean

print str(tipo_int) + " - " + str(type(tipo_int))
print tipo_string + " - " + str(type(tipo_string))
print str(tipo_float) + " - " + str(type(tipo_float))
print str(tipo_bool) + " - " + str(type(tipo_bool))
```

Resultado da execução:

```
100 - <type 'int'>
Testando - <type 'str'>
1.1 - <type 'float'>
False - <type 'bool'>
```

Código-fonte 1 – Tipos de dados
Fonte: Elaborado pelo autor (2021)

O código-fonte “Tipos de dados” define algumas variáveis de cada tipo e então imprime na tela o seu valor e o seu tipo. Muitas das variáveis utilizaram a função **str()** no momento de exibição, com o objetivo de converter o tipo da variável atual para texto (string).

Além de definir as variáveis, é importante saber concatená-las também, a fim de que, em processos automatizados, seja possível realizar certas ações sem precisar gerar lista de palavras predefinidas. No código-fonte “Concatenando variáveis”, são mostradas várias formas de concatenar ou inserir tipos de dados de texto. Com essas informações conseguiremos criar ferramentas voltadas para *fuzzing*.

```
#!/usr/bin/python
# conc.py

a = "Desenvolvimento"
b = " de "
c = "Exploits"

print a + b + c

a = "Desenvolvimento"
a += " de "
a += "Exploits"

print a

a = "Gerando 120x a letra 'A': "
b = "A" * 120

print a + b

Resultado da execução:
Desenvolvimento de Exploits
Desenvolvimento de Exploits
Gerando x120 a letra 'A':
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAA
```

Código-fonte 2 – Concatenando variáveis
Fonte: Elaborado pelo autor (2021)

2.3 Operações matemáticas

Além da manipulação e inserção de texto, é possível utilizar variáveis do tipo inteiras ou flutuantes para fazer operações matemáticas. Veja o código-fonte “Operações matemáticas” com alguns exemplos práticos.

```
#!/usr/bin/python

a = 10
b = 20

print a + b # Soma
print a - b # Subtracao
print a / b # Divisao
print a * b # Multiplicacao

a = 5.5
b = 20.5

print a + b # Soma
print a - b # Subtracao
print a / b # Divisao
print a * b # Multiplicacao

Resultado da execução:
30
-10
0
200
26.0
-15.0
0.268292682927
112.75
```

Código-fonte 3 – Operações matemáticas

Fonte: Elaborado pelo autor (2021)

2.4 Estruturas Condicionais

Raramente utilizaremos estruturas condicionais, mas caso desejamos desenvolver uma prova de conceito com menos chance de haver erros por parte de quem estiver testando, será necessário elaborar alguns eventos específicos para cada condição. Veja no código-fonte “Estrutura condicional” um exemplo que pode ser inserido em um exploit final.

```
#!/usr/bin/python

IP = raw_input()

    if IP:
        print "IP foi inserido"
        if IP == "127.0.0.1":
            print "IP Local"
        elif IP == "8.8.8.8":
            print "IP do Google"
        else:
            print "IP sera verificado"
        else:
            print "IP nao foi inserido"
```

Código-fonte 4 – Estrutura condicional
Fonte: Elaborado pelo autor (2021)

No exemplo do código-fonte “Estrutura condicional” note que ele é iniciado com uma variável **IP**, que será definida a partir do valor que o usuário inserir. Então, a primeira condição é checada e o usuário digita algum valor para o IP ou não. Caso não seja inserido, será mostrado na tela “IP não foi inserido”. Caso seja inserido, entrará em uma outra estrutura condicional, em que verificará o valor do IP com as seguintes condições apresentadas. O **elif** é uma conjunção do **else** e **if**, fazendo com que verifique mais uma vez se a condição é verdadeira ou falsa. Se a condição for falsa, cairá no **else**, uma condição que será utilizada como *escape* caso tudo seja falso.

2.5 Laços de repetição

Laços de repetição são utilizados no intuito de repetir blocos de código. Os principais são: *for* e *while*.

No código-fonte “Estrutura for” é possível ver uma progressão de 0 a 10 caracteres “A”, sendo mostrados na tela.

```
#!/usr/bin/python

for fuzz in range(10):
    print "A" * fuzz
```

Resultado da execução:

```
A
AA
AAA
AAAA
AAAAA
AAAAAA
AAAAAAA
AAAAAAA
AAAAAAA
AAAAAAA
```

Código-fonte 5 – Estrutura for
Fonte: Elaborado pelo autor (2021)

É possível realizar a mesma saída com o *while*. Veja o exemplo.

```
#!/usr/bin/python

a = 0

while a < 10:
    print "A" * a
    a+=1
```

Código-fonte 6 – Estrutura while
Fonte: Elaborado pelo autor (2021)

Caso queira interromper o laço de repetição é possível utilizar o **break**. Já o **continue** é utilizado para continuar no próximo ciclo de repetição do *while*.

2.6 Listas

As listas armazenam valores, como: inteiros, ponto flutuante, texto (strings) e até outras listas.

Exemplo:

```
#!/usr/bin/python

os_linux = ['Ubuntu']

os_linux.append('Debian') # Adicionando item na lista

print os_linux # ['Ubuntu','Debian']

os_linux.insert(0,'Kali') # Inserindo item no índice 0

print os_linux # ['Kali', 'Ubuntu','Debian']

print "Quantidade de itens na lista os_linux: %s" %
(len(os_linux)) # 3

os_linux.remove('Ubuntu') # Remover item 'Ubuntu' da lista

print os_linux # ['Kali','Debian']
```

Resultado da execução:

```
['Ubuntu', 'Debian']
['Kali', 'Ubuntu', 'Debian']
Quantidade de itens na lista os_linux: 3
['Kali', 'Debian']
```

Código-fonte 7 – Listas
Fonte: Elaborado pelo autor (2021)

2.7 Funções

Uma função é um conjunto de instruções que efetua uma tarefa específica. Para criar uma função, utilize a palavra reservada **def**.

Exemplo:

```
#!/usr/bin/python

def conecta():
    print "Conectando na aplicacao"

def conectando(ip,porta):
    print "Conectando atraves de %s:%s" % (ip,porta)

def fuzzing(num):
    for a in range(num):
        cont = "A" * (100*a)
        print "Realizando fuzzing com %i caracteres" % (len(cont))

conecta()

conectando("192.168.0.1",80)

fuzzing(5)
```

Resultado da execução:

```
Conectando na aplicacao
Conectando atraves de 192.168.0.1:80
Realizando fuzzing com 0 caracteres
Realizando fuzzing com 100 caracteres
Realizando fuzzing com 200 caracteres
Realizando fuzzing com 300 caracteres
Realizando fuzzing com 400 caracteres
```

Código-fonte 8 – Funções
Fonte: Elaborado pelo autor (2021)

2.8 Sockets

Sockets podem ser configurados para atuar como um servidor e receber as mensagens ou se conectar a outros aplicativos, como um cliente. Depois que ambas as extremidades de um socket TCP / IP são conectadas, a comunicação é bidirecional.

Exemplo de cliente TCP:

```
#!/usr/bin/python
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(("0.0.0.0", 21))

print s.recv(1024)

s.close()

Resultado da execução:

FTP Server
```

Código-fonte 9 – Cliente TCP
Fonte: Elaborado pelo autor (2021)

Em código-fonte “Servidor TCP”, veja um exemplo de servidor TCP:

```
#!/usr/bin/python
import socket

port = 80

try:
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.bind(("0.0.0.0", port))
    s.listen(5)
    print("Servidor HTTP aberto\r\n")
except:
    print("Falha ao tentar iniciar o servidor")

while True:
    conn, addr = s.accept()
    conn.send('Testando\r\n')
```

Resultado da execução:

Servidor HTTP aberto

Código-fonte 10 – Servidor TCP
Fonte: Elaborado pelo autor (2021)

Usando o **netcat**, mais conhecido como canivete suíço do Kali, é possível se conectar ao servidor TCP criado.

```
root@kali:~# nc 0.0.0.0 80 -v
0.0.0.0: inverse host lookup failed: Unknown host
(UNKNOWN) [0.0.0.0] 80 (http) open
Testing
```

Comando de prompt 3 – Acessando o servidor TCP via *netcat*
Fonte: Elaborado pelo autor (2021)

3 LIDANDO COM VALORES EM BIG ENDIAN E LITTLE ENDIAN

Muitas vezes lidaremos com endereços de memória, que por padrão é no formato big-endian. Porém, ao enviarmos para a pilha, o endereço será invertido de uma maneira que não será interpretada corretamente, ou seja, teremos de lidar com esses tipos de valores antes de enviar para a aplicação.

Veja um exemplo de como realizar essa conversão, utilizando um valor de exemplo 0x01020304 como endereço de memória que está em big endian.

```
#!/usr/bin/python
from struct import *

# Convertendo big endian para little endian

print pack('<L',0x01020304))

# Convertendo little endian para big endian

print pack('L>',0x04030201))
```

Código-fonte 11 – Lidando com endianness

Fonte: Elaborado pelo autor (2021)

Verifique o resultado do big-endian para little endian a partir do **xxd**.

```
root@kali:~# python endianness.py | xxd
00000000: 0403 0201 0a
```

Comando de prompt 4 – Convertendo o big endian para little endian

Fonte: Elaborado pelo autor (2021)

Se visualizar atentamente, é possível observar que a ordem dos bytes foi alterada, iniciando do 04 ao 01.

REFERÊNCIAS

DESENVOLVIMENTO de Exploits. **Sec4us**. [s.d]. Disponível em: <<https://sec4us.com.br/treinamentos/desenvolvimento-de-exploits-32-bits-extended/>>. Acesso em: 18 mar. 2024.

MORENO, D. **Python para Pentest**. São Paulo: Novatec, 2018.

PYTHON: 10 motivos para aprender a linguagem em 2019. **Computer World**. 2019. Disponível em: <<https://www.insightlab.ufc.br/python-10-motivos-para-aprender-a-linguagem-em-2019/>>. Acesso em: 18 mar. 2024.